

Object Oriented Programming with Java

CHAPTER 1

What is a Computer??

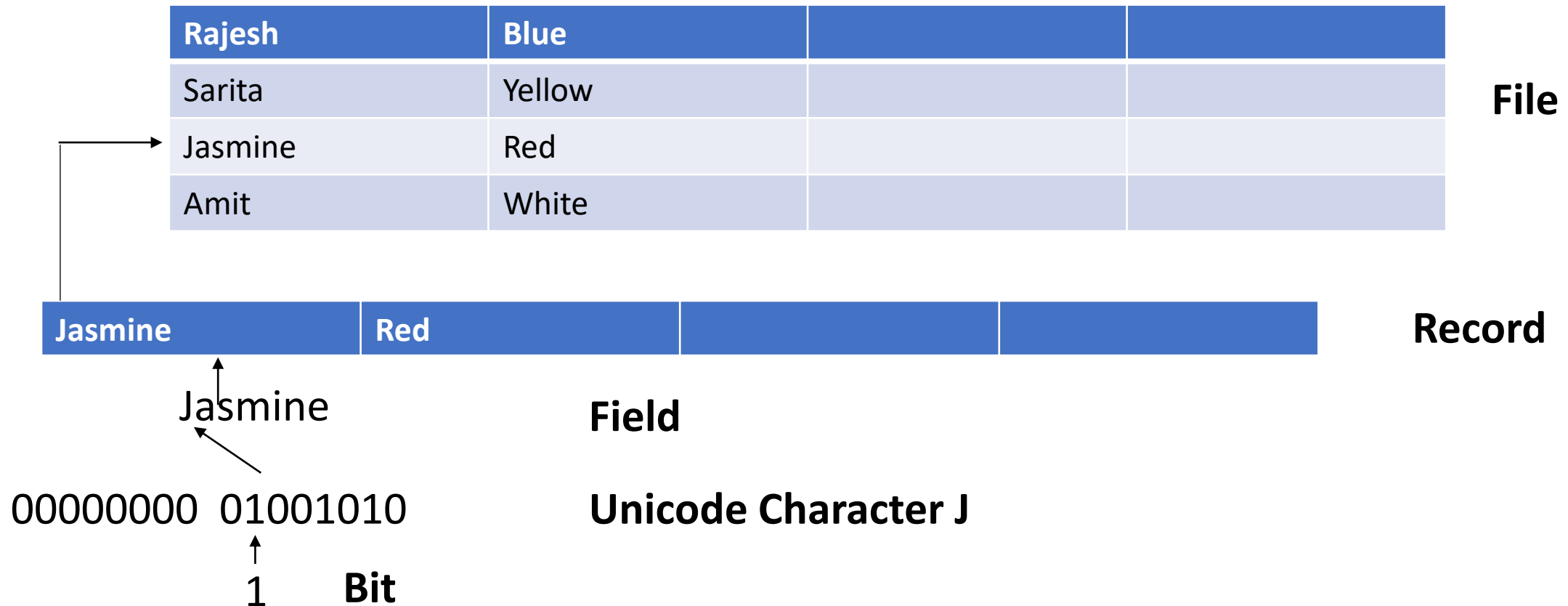
- Computer
 - Device for performing computations and making logical decisions
 - Process *data* using sets of instructions called *computer programs*
- Hardware
 - Devices comprising a computer
 - Keyboard, screen, mouse, disks, memory, CD-ROM, and processing units
- Software
 - Programs that run on a computer

Computer Organization

- Six logical units in every computer
 - **Input unit**
 - Gets information from input devices (keyboard, mouse)
 - **Output unit**
 - Sends information (to screen, to printer, to control other devices)
 - **Memory unit**
 - Rapid access, low capacity, stores input information
 - **Arithmetic and logic unit (ALU)**
 - Arithmetic calculations and logic decisions
 - **Central processing unit (CPU)**
 - Supervises and coordinates sections of the computer
 - **Secondary storage unit**
 - Cheap, long-term, high-capacity storage, stores inactive programs and data

Data Hierarchy

- Data items processed by computers form a **data hierarchy** that becomes larger and more complex in structure as we progress from bits to characters to fields.



Data Hierarchy

- **Bits (Binary Digit):** Smallest data item in computer.
 - Can assume value 0 or 1
- **Characters:** It is tedious to work with data in low-level form of bits.
 - Instead digits (0-9), letters (A-Z and a-z), special symbols (e.g. @, #, \$, %, *, +, -, ", :, ;, , ? and /)
 - Digits, letters and special symbols are known as characters
 - The computer's character set is the set of all characters used to write programs and represent data.
 - Character set represents every character as a pattern of 0s and 1s.
 - Java uses Unicode characters that are composed of two bytes (each eight bits)
 - Unicode contains characters for many of the world's languages

Data Hierarchy

- **Fields:** A field is a group of characters that convey some meaning. Ex. Name, age etc.
- **Records:** Several related fields can be used to compose a record.
 - **Ex. :** In a payroll system, employee can have following fields:
 - Employee ID (Number)
 - Name (String of characters)
 - Address (String of characters)
 - Pay rate (number with decimal point)
- **Files:** A file is a group of related records

Evolution of Operating System

- Batch processing
 - Do only one job or task at a time
- Operating systems
 - Manage transitions between jobs
 - Increased throughput - amount of work computers process
- Multiprogramming
 - Many jobs or tasks *sharing* computer resources
- Timesharing
 - Run small portion of one user's job, move onto next user
 - Programs appear to be running simultaneously

Personal, Distributed and Client/Server Computing

- Personal computing
 - Popularized by Apple Computer in 1977
 - IBM followed suit in 1981 with the IBM Personal Computer
 - Computers economical enough for personal use
 - Stand-alone units
- Distributed computing
 - Organization has a Local Area Network (LAN)
 - Computers linked to it
 - Computing distributed over the LAN

Personal, Distributed and Client/Server Computing

- Client/Server computing
 - File servers offer common programs that client computers access
 - C and C++ popular for writing operating systems, networking, and distributed client/server applications
 - Java used for Internet-based applications
 - Programming in Java can be more productive than C or C++

Machine Languages, Assembly Languages and High-Level Languages

- Types of programming languages

1. Machine languages

- Strings of numbers giving machine specific instructions
- Example:

+1300042774
+1400593419
+1200274027

2. Assembly languages

- English-like abbreviations representing elementary computer operations (translated via assemblers)
- Example:

LOAD	BASEPAY
ADD	OVERPAY
STORE	GROSSPAY

Machine Languages, Assembly Languages and High-Level Languages

- Types of programming languages
 - 3. High-level languages
 - Similar to everyday English and use mathematical notations (translated via compilers)
 - Example:
`grossPay = basePay + overTimePay`

History of Java

- Java
 - Based on C and C++
 - Developed in 1991 for intelligent consumer electronic devices
 - Market did not develop, project in danger of being cancelled
 - Internet exploded in 1993, saved project
 - Used Java to create web pages with dynamic content
 - Java got formally announced in 1995
 - Now used to create web pages with interactive content, enhance web servers, applications for consumer devices (pagers, cell phones)...

Java Class Libraries

- **Java programs**

- Consist of pieces called classes
- Classes contain methods, which perform tasks

- **Class libraries**

- Also known as Java API (Applications Programming Interface)
- Rich collection of predefined classes, which you can use

- **Two parts to learning Java**

- Learning the language itself, so you can create your own classes
- Learning how to use the existing classes in the libraries

Other Higher-Level Languages

- A few other high-level languages have achieved broad acceptance
 - FORTRAN (FORMula TRANslator)
 - Scientific and engineering applications
 - COBOL (COMmon Business Oriented Language)
 - Used to manipulate large amounts of data
 - Pascal
 - Intended for academic use
 - BASIC
 - Developed in 1965
 - Simple language to help novices

Structured Programming

- **Structured programming**
 - Disciplined approach to writing programs
 - Clear, easy to test, debug, and modify
 - Pascal designed to teach structured programming
 - Not used in industrial or commercial applications
- **Multitasking**
 - Many activities run in parallel
 - C and C++ allow one activity at a time
 - Java allows multithreading
 - Activities can occur in parallel

Java Environment

- Java Systems
 - Consist of environment, language, Java Applications Programming Interface (API), class libraries

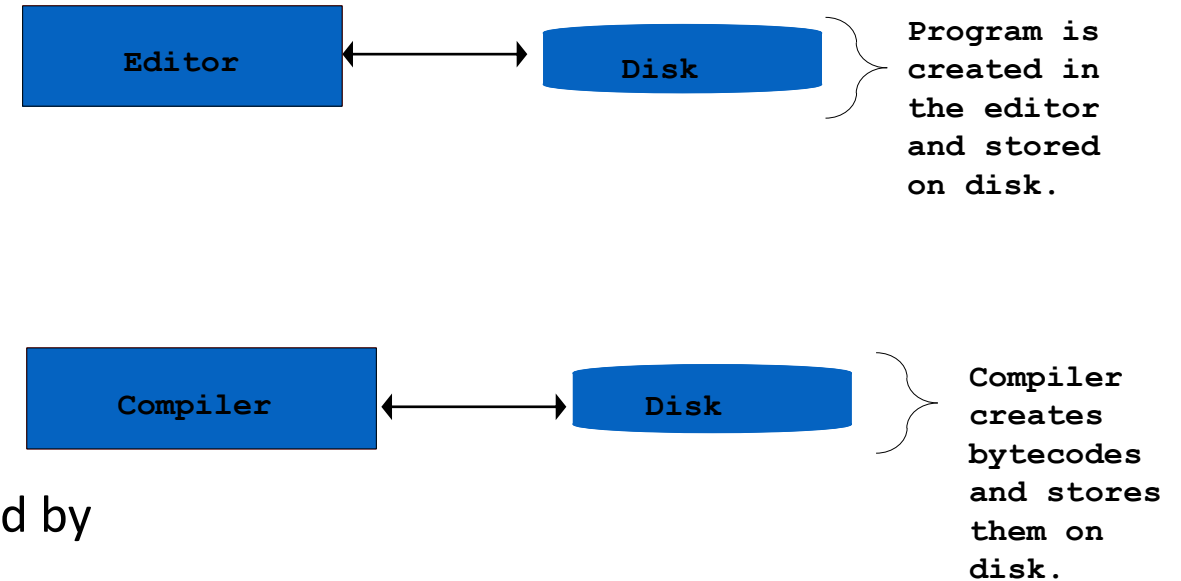
- **Java programs have five phases**

- **Edit**

- Use an editor to type Java program
 - **vi** or **emacs**, notepad, Jbuilder, Visual J++
 - **.java** extension
 - **Example: myProgram.java**

- **Compile**

- Translates program into **bytecodes**, understood by Java interpreter
 - **javac** command: **javac myProgram.java**
 - Creates **.class** file containing bytecodes (**myProgram.class**)

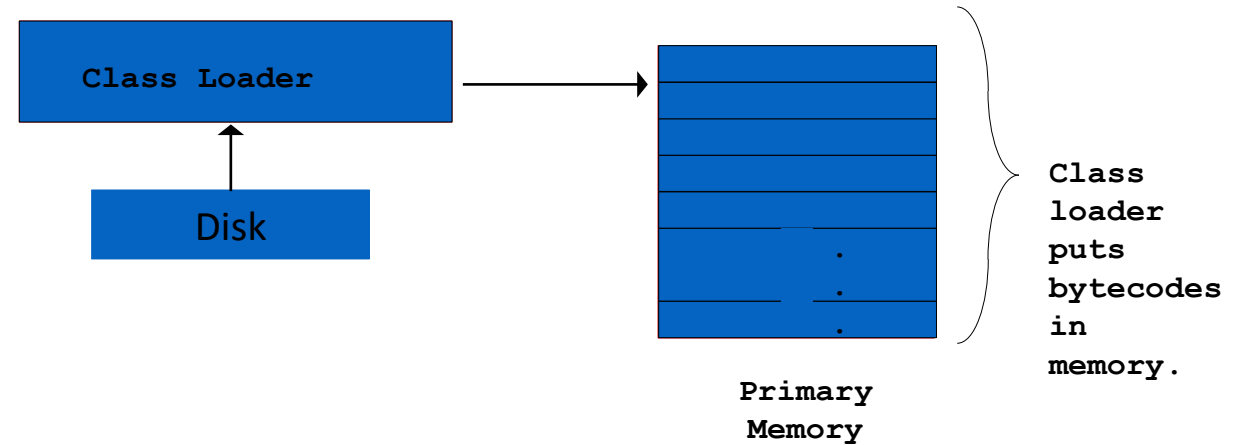


Java Environment

- Java programs have five phases (continued)

- **Loading**

- Class loader transfers **.class** file into memory
 - Applications - run on user's machine
 - Applets - loaded into Web browser, temporary
 - Classes loaded and executed by interpreter with **java** command
>>java myProgram
 - HTML documents can refer to Java Applets, loaded into web browsers
 - To load,
appletviewer Welcome.html
 - **appletviewer** minimal browser, can only interpret applets



Java Environment

- Java programs have five phases (continued)

- **Verify**

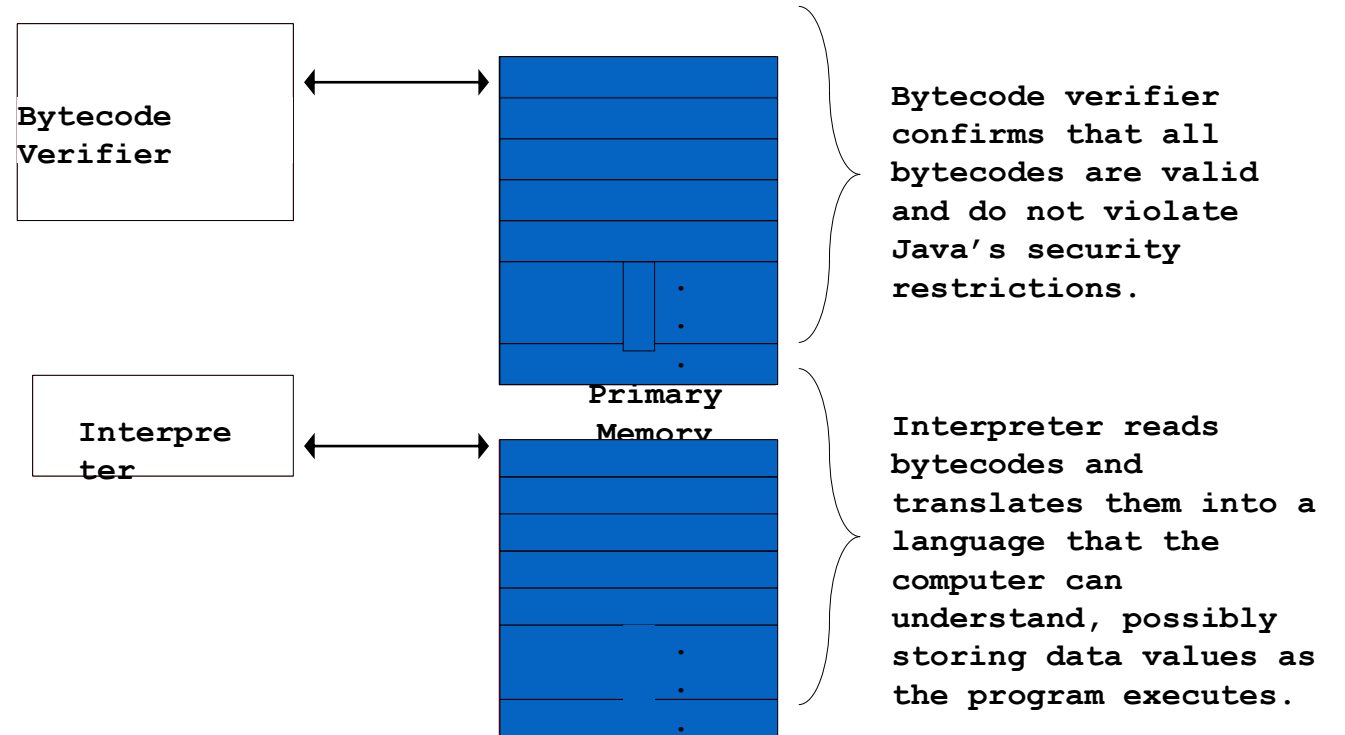
- Bytecode verifier makes sure bytecodes are valid and do not violate security
 - Java must be secure - possible to damage files (viruses)

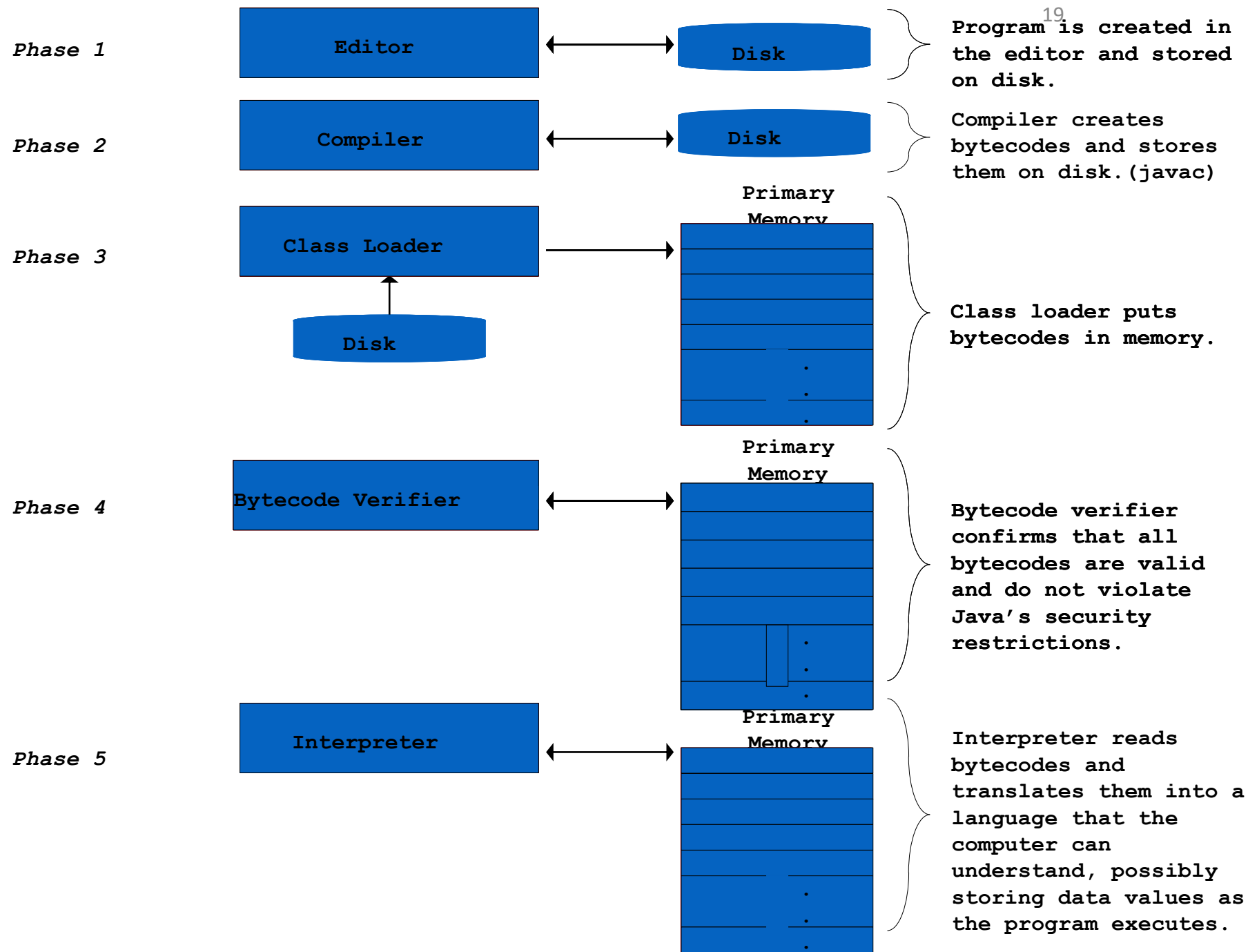
- **Execute**

- Computer interprets program one bytecode at a time
 - Performs actions specified in program

- **Program may not work on first try**

- Make changes in edit phase and repeat





Just-in-time Compiler

- Just-in-time compiler
 - Midway between compiling and interpreting
 - As interpreter runs, compiles code and executes it
 - Not as efficient as full compilers
 - Being developed for Java
 - Integrated Development Environment (IDE)
 - Tools to support software development
 - Several Java IDE's are as powerful as C / C++ IDE's

Syllabus: Chapter 1 (continued..)

(Introduction to Java Applications)

Outline

- 2.1 Introduction
- 2.2 A Simple Program: Printing a Line of Text
- 2.3 Another Java Application: Adding Integers
- 2.4 Memory Concepts
- 2.5 Arithmetic
- 2.6 Decision Making: Equality and Relational Operators

A Simple Program: Printing a Line of Text

```
1 // Fig. 2.1: Welcome1.java
2 // A first program in Java
3
4 public class Welcome1 {
5     public static void main( String args[] )
6     {
7         System.out.println( "Welcome to Java
8     }
9 }
```

Welcome to Java Programming!

2.2 A Simple Program: Printing a Line of Text

```
1 // Fig. 2.1: Welcome1.java
```

- **Comments**

- `//` remainder of line is comment
 - Comments are ignored by the compiler
 - Used for documentation and code description
- Multiple line comments: `/* ... */`
`/* This is a multiple
line comment. It can
be split over many lines */`

```
2 // A first program in Java
```

- Another line of comments

2.2 A Simple Program: Printing a Line of Text

3

- Blank line
 - Makes program more readable
 - Blank lines, spaces, and tabs are *whitespace characters*
 - Ignored by compiler

4 **public class Welcome1 {**

- Begins class definition for class **Welcome1**
 - Every Java program has at least one user-defined class
 - Keyword: words reserved for use by Java
 - **class** keyword followed by class name
 - Naming classes: capitalize every word
 - **SampleClassName**

2.2 A Simple Program: Printing a Line of Text

```
4 public class Welcome1 {
```

- Name of class is called *identifier*
 - Series of characters consisting of letters, digits, underscores (`_`) and dollar signs (`$`)
 - Does not begin with a digit, has no spaces
 - Examples: **Welcome1**, **\$value**, **_value**, **button7**
 - **7button** is invalid
 - Case sensitive (capitalization matters)
 - **a1** and **A1** are different
- use **public** keyword
 - Certain details not important now (discussed later)

2.2 A Simple Program: Printing a Line of Text

```
4 public class Welcome1 {
```

- Saving files
 - File name is class name and **.java** extension
 - **Welcome1.java**
- Left brace {
 - Begins body of every class
 - Right brace ends definition (line 9)

```
5     public static void main( String args[] )
```

- Part of every Java application
 - Applications begin executing at **main**
 - Parenthesis indicate **main** is a method
 - Java applications contain one or more methods

A Simple Program: Printing a Line of Text

```
5 public static void main( String args[] )
```

- Exactly one method must be called **main**
- Methods can perform tasks and return information
 - **void** means **main** returns no information

```
6     {
```

- Left brace begins body of method definition
 - Ended by right brace

2.2 A Simple Program: Printing a Line of Text

```
7 System.out.println("Welcome to Java Programming!");
```

- Instructs computer to perform an action
 - Prints string of characters
 - String - series characters inside double quotes
 - Whitespaces in strings are not ignored by compiler
- **System.out**
 - Standard output object
 - Print to command window (i.e., MS-DOS prompt)
- Method **System.out.println**
 - Displays line of text
 - Text : Argument inside parenthesis
- This line is known as a statement
 - Statements must end with semicolon ;

2.2 A Simple Program: Printing a Line of Text

```
8      }
```

- Ends method definition

```
9    }
```
- Ends class definition
- Can add comments to keep track of ending braces
- Lines 8 and 9 could be rewritten as:

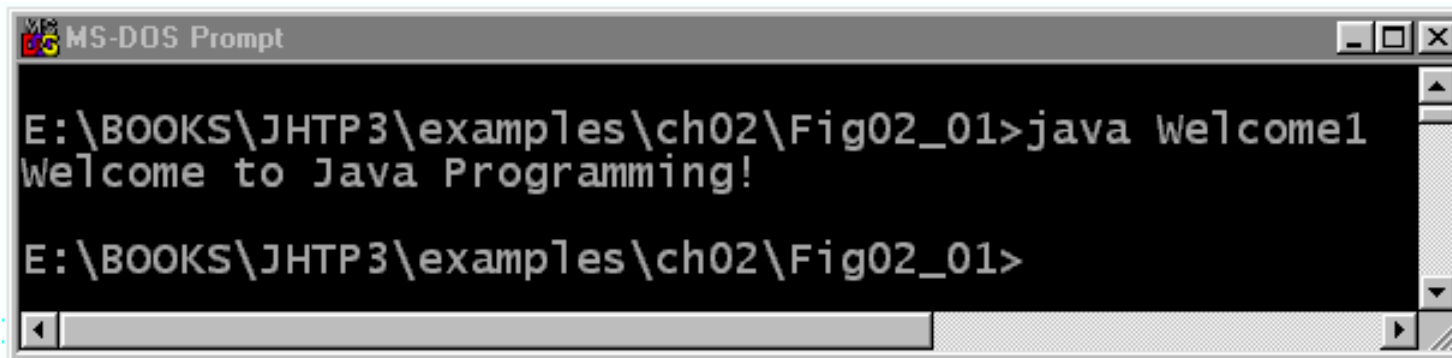
```
8      }    // end of method main()  
9    }      // end of class Welcome1
```

- Remember, compiler ignores comments

2.2 A Simple Program: Printing a Line of Text

- Compiling a program
 - Open a command window, go to directory where program is stored
 - Type **javac Welcome1.java**
 - If no errors, **Welcome1.class** created
 - Has bytecodes that represent application
 - Bytecodes passed to Java interpreter
- Executing a program
 - Type **java Welcome1**
 - Interpreter loads **.class** file for class **Welcome1**
 - **.class** extension omitted from command
 - Interpreter calls method **main**

```
1 // Fig. 2.1: Welcome1.java
2 // A first program in Java
3
4 public class Welcome1 {
5     public static void main( String args[] )
6     {
7         System.out.println( "Welcome to Java
8     }
9 }
```



The screenshot shows a classic MS-DOS command prompt window. The title bar reads "MS-DOS Prompt". The command prompt shows the current directory as "E:\BOOKS\JHTP3\examples\ch02\Fig02_01". The user has entered the command "java Welcome1", and the output displayed is "Welcome to Java Programming!". The prompt is ready for the next command.

```
MS-DOS Prompt
E:\BOOKS\JHTP3\examples\ch02\Fig02_01>java Welcome1
Welcome to Java Programming!
E:\BOOKS\JHTP3\examples\ch02\Fig02_01>
```

2.2 A Simple Program: Printing a Line of Text

- Other methods
 - **`System.out.println`**
 - Prints argument, puts cursor on new line
 - **`System.out.print`**
 - Prints argument, keeps cursor on same line


```
1 // Fig. 2.3: Welcome2.java
2 // Printing a line with multiple statements
3
4 public class Welcome2 {
5     public static void main( String args[] )
6     {
7         System.out.print( "Welcome to " );
8         System.out.println( "Java Programming!" );
9     }
10 }
```



System.out.print keeps the cursor on the same line, so **System.out.println** continues on the same line.

Welcome to Java Programming!

2.2 A Simple Program: Printing a Line of Text

- Escape characters
 - Backslash (\)
 - Indicates special characters be output
 - Backslash combined with character makes escape sequence
 - `\n` - newline `\r` - carriage return `\"` - double quote
 - `\t` - tab `\\` - backslash
- Usage
 - Can use in **`System.out.println`** or **`System.out.print`** to create new lines
 - `System.out.println( "Welcome\nto\nJava\nProgramming!" );`

```
1 // Fig. 2.4: Welcome3.java
2 // Printing multiple lines with a single statement
3
4 public class Welcome3 {
5     public static void main( String args[] )
6     {
7         System.out.println( "Welcome\nto\nJava\nProgramming!" );
8     }
9 }
```

```
Welcome
to
Java
Programming!
```

Notice how a new line is output for each `\n` escape sequence.

Data Types

- Integers: this group includes **byte, short, int and long** which are for whole valued signed numbers.
- Floating point numbers: This group includes **float and double** which represent numbers with fractional precision.
- Character: This group includes **char** which represents symbols in a character set, like letters and numbers.
- Boolean: This group includes **boolean**, which is a special type for representing true/false values.

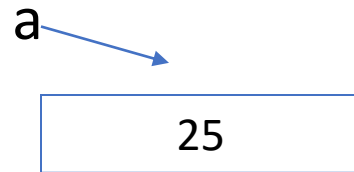
Variable

- A variable is a location in computer's memory where a value can be stored for later use
- All Java variables must be declared with a name and type of variable before using
- A variable's name enables the program to access the value stored in its memory location
- A variable's name can be any valid identifier
- Variable's type specifies what kind of information can be stored in that memory location

Variable Declaration

`int a;` Declares integer that can hold integer value and is not yet initialized (not given value). Range of integer: -2,147,483,648 to + 2,147,483,647

`a = 25;`



Declaring 2 variables on same line:

`int a, b;`

Initializing variable when it is declared

`int a = 25, b = 0;`

Variable Declaration

- Variable declaration of other data types

float salary;

long Emp_ID;

double yearly_expenses;

- **Primitive data types:**

- boolean, byte, char, short, int, long, float and double

Declaring and Using Scanner Object

- Scanner enables a program to read data from input device
- Input source can be: keyboard, file on disk etc.
- Before using it, its object has to be created and the source of input should be specified:
 - `Scanner input = new Scanner (System.in);`
- The statement uses **new** keyword to create new object of Scanner.
- **System.in:** it is standard input object that enables reading bytes of information typed by user
- **The Scanner translates** these bytes into type (like int) required in program

2.5 Another Java Application: Adding Integers

- Variables
 - Location in memory that stores a value
 - Declare with name and data type before use
 - **firstNumber** and **secondNumber** are of data type **String** (package **java.lang**)
 - Hold strings
 - Variable name: any valid identifier
 - Declarations end with semicolons ;
 - Can declare multiple variables of the same type at a time
 - Use comma separated list
- Can add comments to describe purpose of variables

2.3 Another Java Application: Adding Integers

```
11      int number1,          // first number to add
12          number2,          // second number to add
13          sum;               // sum of number1 and number2
```

- Declares variables **number1**, **number2**, and **sum** of type **int**
 - **int** holds integer values (whole numbers): i.e., 0, -4, 97
 - Data types **float** and **double** can hold decimal numbers
 - Data type **char** can hold a single character
 - Primitive data types - more Chapter 4

Memory Concepts

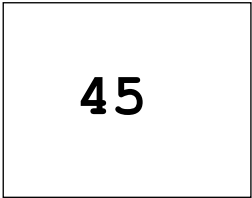
- Variables
 - Every variable has a *name*, a *type*, a *size* and a *value*
 - Name corresponds to *location* in memory
 - When new value is placed into a variable, replaces (and destroys) previous value
 - Reading variables from memory does not change them
- Visual representation

```
int number1;
```

```
number1 = 45;
```

number1

45



Arithmetic

- Arithmetic calculations are used in most programs

- Usage

- $*$ for multiplication
- $/$ for division
- $+$, $-$
- No operator for exponentiation
- Integer division truncates remainder

$7 / 5$ evaluates to 1

- Modulus operator $\%$ returns the remainder

$7 \% 5$ evaluates to 2

Arithmetic

- Operator precedence
 - Some arithmetic operators act before others (i.e., multiplication before addition)
 - Use parenthesis when needed
 - Example: Find the average of three variables **a**, **b** and **c**
 - Do not use: **a + b + c / 3**
 - Use: **(a + b + c) / 3**

Arithmetic

Operator(s)	Operation(s)	Order of evaluation (precedence)
()	Parentheses	Evaluated first. If the parentheses are nested, the expression in the innermost pair is evaluated first. If there are several pairs of parentheses "on the same level" (i.e., not nested), they are evaluated left to right.
*, /, or %	Multiplication Division Modulus	Evaluated second. If there are several, they are evaluated left to right.
+ or -	Addition Subtraction	Evaluated last. If there are several, they are evaluated left to right.

2.6 Decision Making: Equality and Relational Operators

- **if** control structure
 - Simple version in this section, more detail later
 - If a condition is true, then the body of the **if** statement executed
 - 0 interpreted as false, non-zero is true
 - Control always resumes after the **if** structure
 - Conditions for **if** structures can be formed using equality or relational operators (next slide)

```
if ( condition ) {  
    statement executed if condition true  
}
```

- No semicolon needed after condition

Decision Making: Equality and Relational Operators

Standard algebraic equality operator or relational operator	Java equality or relational operator	Example of Java condition	Meaning of Java Condition
Relational operators			
>	>	x > y	x is greater than y
<	<	x < y	x is less than y
>=	>=	x >= y	x is greater than or equal to y
<=	<=	x <= y	x is less than or equal to y
Equality operators			
=	==	x == y	x is equal to y
≠	!=	x != y	x is not equal to y


```
public class Comparison {  
    public static void main (String args[])  
    {  
        int first_number;  
        int second_number;  
        Scanner input = new Scanner(System.in);  
  
        System.out.println("Enter First Number:");  
        first_number = input.nextInt();  
        second_number = input.nextInt();  
  
        if(first_number == second_number)  
            System.out.println("First number " + first_number + " is  
equal to second          number " + second_number);  
    }  
}
```

```
if(first_number != second_number)
    System.out.println("First number " +
        first_number + " is not equal to second number "
        + second_number);

if(first_number < second_number)
    System.out.println("First number " +
        first_number
        + " is less than second number " + second_number);

if(first_number > second_number)
    System.out.println("First number " +
        first_number + " is
        greater than second number " + second_number);
}
```